# Strict aliasing in C++

## and some other memory topics

**Author**: Damir Ljubic
**email**: damirlj@yahoo.com

## Introduction

What is strict aliasing in C++?
Well, we can answer it by introducing two short definitions:

- **Aliasing** means that we have two (or more) expressions referring to the same memory location. It's about having two (or more) different representations of the same memory layout.
- **Strict aliasing** means that compiler may assume that two (or more) pointers to unrelated - incompatible types never alias. Standard brings certain rules on top of it – restrictions based on which the first definition holds

Accessing an object's stored value is UB (Undefined Behavior) if the **glvalue** (generalized value) of the accessing object's static type is incompatible with the stored object's dynamic type.
Precisely, the static type needs to be either:

- **The same as** dynamic type
- Similar to the dynamic type: where "similar" means differs only in **cv qualifiers** at some level
- Corresponding **sing/unsigned type** (arithmetic types)
- In case of the inheritance, **base class of the dynamic type**
- **Aggregates** and **unions** containing that type as the member

**// NOK example**

```
int a = 11; // glvalue representation of the object
auto p = reinterpret_cast<float*>(&a); // convertible to the static (float) type - but not the same or similar to it
*p = 5.3f; // UB!
```

What we have here is *type punning* – stealing the type's identity, replacing it with some other type. It's act of crime – would you agree?
C++20 offers *std::bit_cast*<> which should make this possible, since both types are scalar – arithmetic types (trivially copyable) of the same, 4 bytes size.

**// OK example**

```
int a = -1; // glvalue representation of the object
auto p = reinterpret_cast<unsigned int*>(&a); // corresponding unsigned version
*p = 4;
```

- This also applies to the **byte-like types**, since any type can be read as an "raw bytes" array, while otherwise in not the case. We will see shortly – in custom allocator example

**// OK example**

```
static_assert(std::endian::native == std::endian::little);
int a = 11;
auto bytes = reinterpret_cast<std::byte*>(&a);
bytes[0] &= std::byte{0xFE};
assert (a == 10);
```

## Custom allocators

For creating custom allocators, especially those that maintain internal memory storage on the stack, we use raw (uninitialized) bytes, to create an arbitrary type T.
Any type-aware allocation is two steps operation:

- Memory allocations (preallocated memory on the stack)
- Memory initialization (constructing the instance of T into memory): T becomes our **dynamic type** – no aliasing involved!

```
alignas(T) std::byte buff[SLOTS * sizeof(T)]; // internal memory storage on stack

auto mem = reinterpret_cast<T*>(&buff[slot * sizeof(T)]);
auto p = std::construct_at(mem, std::forward<Args>(args)…);
```

And now – when the memory is initialized for holding the concrete object of type T, we can safely access it.

A general note

| |
|---|
| Forcibly casting the raw memory into some other type, with precondition that the type's footprint fits into the memory – calling the *reinterpret_cast*<> is not a priori UB. Accessing afterwards non-initialized memory, on the other hand, is undefined behavior |

In addition to this, C++23 introduced the *std::start_lifetime_as<T>/ std::start_lifetime_as_array<T>* as the way to use uninitialized memory as if the T is created – already constructed there. This applies only for implicit-lifetime types: for trivially copyable types (scalar, POD, aggregate – all types that are member-wise copyable and have a trivial default constructor/ trivial destructor).

A bonus point.
For the custom allocators that are type-erased, the type-aware allocation may look like this

```
template <typename T, typename... Args>
   requires (std::is_constructible_v<T, Args...> and
                sizeof(T) <= BlockSize and alignof(T) <= Alignment)
[[nodiscard]] T* allocate(Args&&... args)
   noexcept(std::is_nothrow_constructible_v<T, Args...>)
{
      if (free_list_ == nullptr) return nullptr; // no free blocks available
      auto* node = free_list_;
      free_list_ = node->next;
      auto* mem = reinterpret_cast<T*>(node);
      T* ptr = std::construct_at(mem, std::forward<Args>(args)...);
      ++occupied_slots_;
      return ptr;
}
```

The *std::launder* is another memory-related feature that serves to instruct the compiler not to take any assumptions on the given pointer – to reevaluate it again.
Its own precondition demands that an object of a type "**similar**" to T — the same type up to cv-qualification — **is already alive at that address**.
Therefore, using the

```
auto* mem = std::launder(reinterpret_cast<T*>(node));
```

would we wrong here – since the "live" object in the memory is Node, not T.

The genuinely *useful* case is when you cannot keep the returned pointer — re-deriving a pointer to an already-living object out of buff_:

```
// a block known to be on the free list: it holds a live Node
[[nodiscard]] Node* node_at(std::size_t block) noexcept
{
   return std::launder(reinterpret_cast<Node*>(&buff_[block * BlockSize]));
}
```